



INNOMATICS
RESEARCH LABS

Deliverable 1: High-Level Design (HLD)

Name : Kandela Vamshi

Intern Id : IN326013704

Mail: Vamshikandela29@gmail.com

RAG-Based Customer Support Assistant With LangGraph & HITL

1. System Overview

I. Problem Statement

Customer support systems often rely on static FAQs or manual agents, leading to delayed responses and limited scalability. There is a need for an intelligent system that can automatically answer user queries based on available knowledge while handling uncertainty effectively.

II. Objective

The objective of this system is to build a Retrieval-Augmented Generation (RAG)-based customer support assistant that:

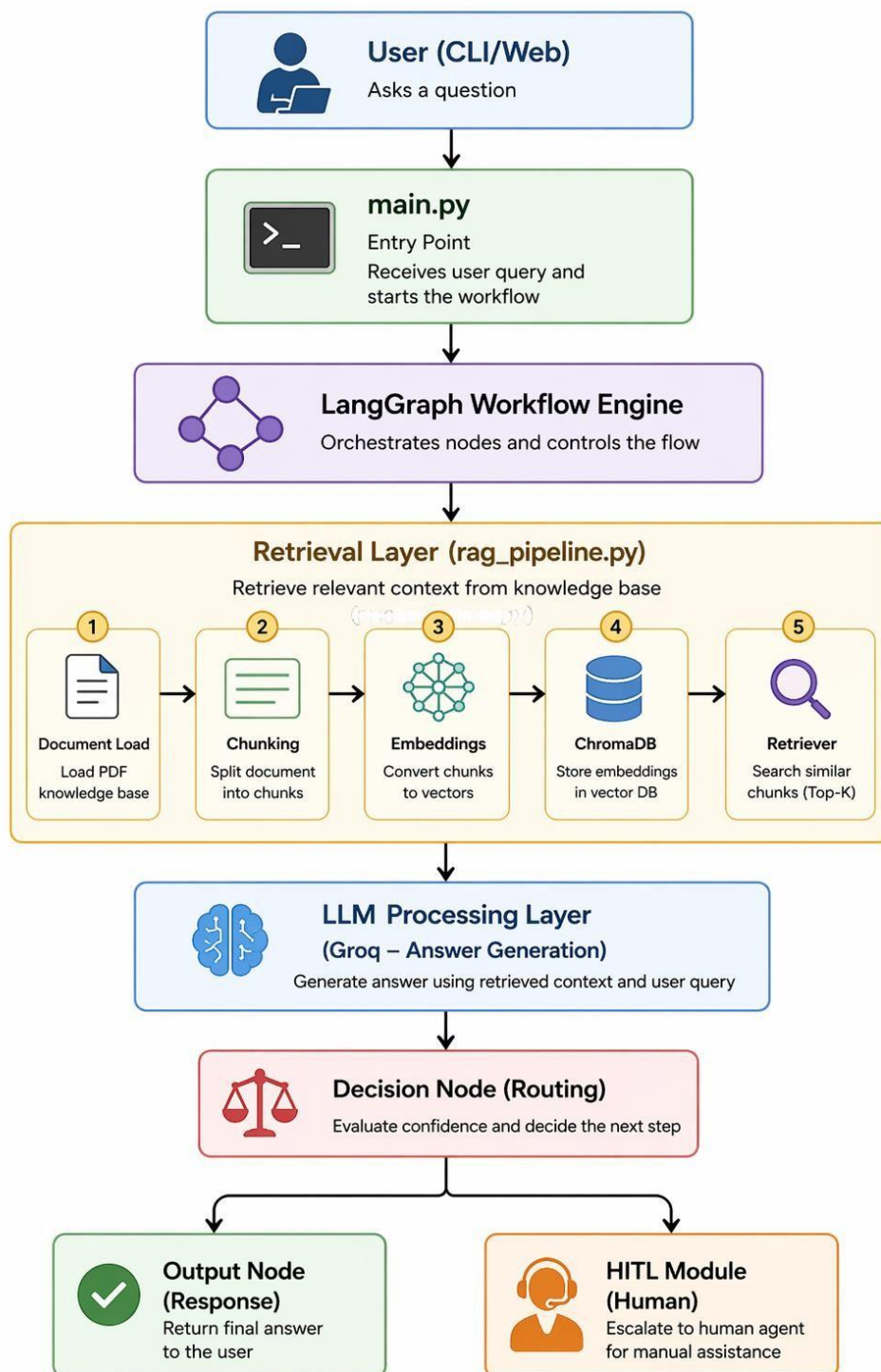
- Processes a PDF knowledge base
- Retrieves relevant information using embeddings
- Generates context-aware responses using an LLM
- Uses a graph-based workflow for decision-making
- Escalates queries to a human agent when confidence is low

III. Scope

- Single PDF knowledge base
- CLI-based interaction
- Semantic retrieval using embeddings
- Conditional routing (answer vs escalation)

Human-in-the-loop fallback

2. System Architecture Diagram



3. Component Description

1. Document Loader

Loads the PDF knowledge base using PyPDFLoader and extracts raw text content.

2. Chunking Strategy

Splits the document into smaller chunks using a recursive text splitter to improve retrieval accuracy and context handling.

3. Embedding Model

Converts text chunks into vector representations using HuggingFace embeddings for semantic similarity search. **4. Vector Store (ChromaDB)**

Stores embeddings locally and enables efficient similarity-based retrieval.

5. Retriever

Fetches top-k relevant chunks based on query similarity using vector search.

6. LLM (Groq)

Generates final answers using retrieved context and user query.

7. Graph Workflow Engine (LangGraph)

Controls execution flow using nodes and edges, enabling conditional routing and modular processing.

8. Routing Layer

Determines whether to:

- Return the generated answer
- Escalate to a human agent **9.**

HITL Module (Human-in-the-Loop)

Handles escalation when:

- Confidence is low
- Answer is insufficient

Provides a fallback response simulating human intervention.

4. Data Flow

Document Processing Flow

1. Load PDF
2. Split into chunks

3. Generate embeddings
4. Store in ChromaDB

Query Processing Flow

1. User inputs query
 2. Retrieve relevant chunks from vector store
 3. Send query + context to LLM
 4. Generate answer
 5. Calculate confidence score
 6. Decision:
 - High confidence → return answer
 - Low confidence → escalate to human
-

5. Technology Choices

ChromaDB

- Lightweight and local vector database
- Easy integration with LangChain
- Suitable for small to medium-scale applications

LangGraph

- Enables graph-based workflow design
- Supports conditional routing and modular nodes
- Improves system control and scalability

Groq LLM

- Fast inference speed
- Cost-efficient
- Suitable for real-time query handling

HuggingFace Embeddings

- Open-source and reliable

- Good semantic understanding
 - Easy integration
-

6. Scalability Considerations

Handling Large Documents

- Use optimized chunking strategies
- Increase vector store capacity
- Implement indexing improvements

Handling High Query Load

- Introduce caching mechanisms
- Use asynchronous processing
- Scale vector database

Latency Optimization

- Reduce number of retrieved chunks (TOP_K tuning)
- Use faster embedding models
- Optimize LLM calls

Future Scalability Enhancements

- Multi-document support
 - Distributed vector storage
 - Load balancing for LLM requests
-

7. Summary

This system demonstrates a modular and scalable RAG-based architecture that integrates document retrieval, LLM-based answer generation, and human fallback mechanisms. The use of LangGraph enables structured workflow management, while ChromaDB and embeddings provide efficient semantic search capabilities.
